A PROJECT REPORT ON

rainmaker-rs: Enabling Type Safe IoT Systems using Rust

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Akshay Lahoti	Exam No: B400050468
Chinmay Dixit	Exam No: B400050383
Shreyash Bubane	Exam No: B400050350

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2021-2022

CERTIFICATE

This is to certify that the project report entitled

rainmaker-rs

Submitted by

Akshay Lahoti	Exam No: B400050468
Chinmay Dixit	Exam No: B400050383
Shreyash Bubane	Exam No: B400050350

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. R.S.Paswan** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. R. S. Paswan
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'rainmekerrs'****.*

We would like to take this opportunity to thank ***Prof. R. S. Paswan*** *giving us all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

We are also grateful to ***Dr G. V. Kale****, Head of Computer Engineering Department, PICT for her indispensable support, suggestions.*

In the end our special thanks to ***Dr. Mukta S. Takalikar****, and* ***Mr. Kedar Sovani****, Espressif Systems Pvt. Ltd. and all faculty members for their whole hearted cooperation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.*

Akshay Lahoti
Chinmay Dixit
Shreyash Bubane
(B.E. Computer Engg.)

ABSTRACT

This project tries to build an open source IoT home automation framework cross-compatible with ESP RainMaker. This will enable IoT developers to develop applications utilizing the memory safety guarantees and robust tooling of Rust programming language and use the infrastructure of ESP RainMaker.

ESP RainMaker is an end-to-end AIoT solution developed by Espressif Systems for home automation, enabling device control via mobile applications. However, its implementation in procedural C lacks memory safety guarantees and is limited to Espressif's hardware. rainmaker-rs is an open-source, cross-platform Rust-based reimplementation of ESP RainMaker, targeting both ESP-IDF and Linux platforms while maintaining compatibility with the existing RainMaker ecosystem. By leveraging Rust's safety features—including memory safety, ownership model, and a strong type system, rainmaker-rs enhances reliability and security in IoT firmware development. It supports the creation of smart home devices such as switches and lights, with integration into Google Home and Amazon Alexa, while also enabling remote control over both the internet and local networks. Furthermore, the architecture of rainmaker-rs extends beyond home automation, providing a robust foundation for the development of various remotely controlled applications.

Features:

- Remote control (controlling devices from anywhere over internet)
- Home Assistant Integration
- WiFi Provisioning (Identifies nearby APs which can be configured using phone application)
- Device Automations
- Local Control (control devices on same WiFi network directly with near zero delay)

Contents

List of Figures

List of Tables

CHAPTER 1
INTRODUCTION

1.1 Overview

RainMaker-rs is an open-source Rust implementation of Espressif's RainMaker platform, designed to provide a unified framework for developing, managing, and controlling smart devices. Unlike the original implementation, RainMaker-rs runs not only on ESP32 microcontrollers but also on Linux systems, enabling developers to build and test IoT applications on multiple platforms. The project is hosted on GitHub at `https://github.com/rainmaker-rs/rainmaker`, with supporting abstraction layers in `https://github.com/rainmaker-rs/components`, and a project starter template at `https://github.com/rainmaker-rs/template`.

1.2 Motivation

The motivation behind RainMaker-rs is to bring the performance, safety, and modern concurrency features of Rust into the IoT ecosystem, particularly for ESP32-based projects. While Espressif's native SDK is powerful, it is C-based and prone to memory safety issues. By adopting Rust, RainMaker-rs aims to reduce bugs, improve maintainability, and make it easier for developers to write cross-platform code with strong guarantees.

1.3 Problem Definition and Objectives

The main objective of RainMaker-rs is to provide a Rust-based alternative to the official RainMaker SDK, while maintaining feature parity and extending compatibility to Linux systems. The problem it addresses is the lack of a memory-safe, type-safe, and modern SDK for ESP32 and IoT systems. Specific objectives include:

- Designing a modular framework with clear abstractions for devices, nodes, and parameters.
- Providing an architecture that runs on both ESP32 and Linux platforms.
- Facilitating remote device management, monitoring, and control through protocols like MQTT.
- Offering developers a ready-to-use template to bootstrap their own IoT projects.

1.4 Project Scope & Limitations

The scope of RainMaker-rs includes:

- Cross-platform abstraction layers for ESP32 and Linux.
- Support for typical RainMaker use cases like smart switches, sensors, and actuators.
- Integration with communication protocols such as MQTT.

Limitations:

- Hardware-specific features may only be available on ESP32.
- Some advanced features from Espressif's C SDK may not yet be ported.
- The Linux implementation is primarily intended for development and simulation, not for production-level device deployment.

1.5 Methodologies of Problem Solving

The development of RainMaker-rs follows a modular and incremental approach:

- Abstraction design: Define clear trait-based interfaces for hardware-specific and platform-independent components.
- Cross-platform development: Use conditional compilation to target both ESP32 and Linux environments.
- Community-driven development: Publish code on GitHub, encourage contributions, and maintain comprehensive documentation.
- Continuous integration: Apply CI workflows to ensure build and test consistency across platforms.

CHAPTER 2
LITERATURE SURVEY

The field of Internet of Things (IoT) has seen rapid growth in recent years, with an increasing demand for robust, secure, and scalable device management frameworks. Espressif's RainMaker is one such platform designed to simplify the development and deployment of ESP32-based smart devices. RainMaker provides a cloud platform, device firmware, and mobile applications to help developers build connected devices efficiently.

2.1 Espressif RainMaker

Espressif RainMaker is a comprehensive IoT solution that enables developers to create, manage, and control connected devices using ESP32 microcontrollers. It provides essential features such as device provisioning, control through mobile apps, cloud synchronization, and integration with popular home automation systems. RainMaker's reference implementation, however, is primarily written in C and tightly coupled to the ESP-IDF framework.

2.2 Existing Home Automation Frameworks

Several open-source projects have emerged to simplify IoT development:

- **Tasmota**: A firmware based on ESP8266/ESP32 that enables devices to connect to home automation platforms such as Home Assistant.
- **ESPHome**: Another ESP8266/ESP32-based project that allows easy configuration of devices using YAML and integrates with Home Assistant.
- **OpenHAB** and **Home Assistant**: Software platforms that integrate various smart devices and protocols under a single management system.

While these frameworks provide excellent hardware and software support, they often rely on dynamic languages or C-based implementations, which may introduce memory safety issues.

2.3 Rust in Embedded and Systems Programming

Rust has gained significant attention in the systems programming community due to its strong memory safety guarantees, zero-cost abstractions,

and modern concurrency support. In the embedded space, Rust is being adopted through frameworks like `embedded-hal`, `smoltcp`, and `RTIC` (Real-Time Interrupt-driven Concurrency). Rust's ecosystem makes it possible to write firmware that is safe, performant, and less prone to common errors like null pointer dereferencing or data races.

2.4 RainMaker-rs: Bridging Rust and IoT

RainMaker-rs builds on the strengths of Rust to provide an alternative implementation of Espressif's RainMaker platform. By offering abstraction layers for ESP32 and Linux, it enables:

- Cross-platform development and testing.
- Improved safety and reliability compared to traditional C-based frameworks.
- A modular and extensible architecture that is easier to maintain and scale.

The RainMaker-rs project is positioned as a novel contribution in the IoT ecosystem, combining Rust's safety and performance with the flexibility of Espressif's hardware.

2.5 Comparison with Related Work

Compared to existing firmware solutions like Tasmota and ESPHome, RainMaker-rs offers:

- A developer-friendly Rust API.
- Strong compile-time guarantees that reduce runtime errors.
- Support for Linux as a development and simulation target, extending beyond microcontroller-only environments.

This positions RainMaker-rs as a promising platform for developers seeking a modern, type-safe approach to IoT device development.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

- Developers have prior experience with Rust and embedded systems.
- The ESP32 platform uses the ESP-IDF toolchain.
- Linux targets require Rust with standard library support.
- Internet connectivity is available for cloud services such as MQTT brokers.
- Developers have access to GitHub repositories: `https://github.com/rainmaker-rs/rainmaker`, `https://github.com/rainmaker-rs/components`, `https://github.com/rainmaker-rs/template`.

3.2 Functional Requirements

3.2.1 System Feature 1: Device Abstraction

- Provide trait-based abstractions for devices, nodes, and parameters.
- Allow seamless development for ESP32 and Linux using conditional compilation.

3.2.2 System Feature 2: Communication and Control

- Enable communication using MQTT.
- Provide a mechanism for receiving and handling device commands.
- Support remote monitoring and control.

3.2.3 System Feature 3: Template Project

- Provide a starter template for new RainMaker-rs projects.
- Include examples for both ESP32 and Linux targets.

3.3 External Interface Requirements

3.3.1 User Interfaces

- Command-line interface (CLI) for building, flashing, and monitoring devices.
- Compatibility with RainMaker mobile app for device control.

3.3.2 Hardware Interfaces

- Support for ESP32 microcontrollers with Wi-Fi capability.
- GPIO, sensors, and actuators as applicable.

3.3.3 Software Interfaces

- Integration with ESP-IDF on ESP32.
- Use of Rust standard libraries and crates on Linux.

3.3.4 Communication Interfaces

- MQTT for cloud communication.
- Wi-Fi for device connectivity.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- Low-latency communication between devices and cloud.
- Efficient resource usage on constrained hardware.

3.4.2 Safety Requirements

- Safe concurrent access to hardware resources.
- Prevent memory corruption through Rust's ownership model.

3.4.3 Security Requirements

- Secure MQTT communication (TLS support).
- Protection against unauthorized device access.

3.4.4 Software Quality Attributes

- Modularity and extensibility.
- Maintainable and well-documented codebase.

3.5 System Requirements

3.5.1 Database Requirements

- No centralized database; state is managed locally or via cloud (MQTT).

3.5.2 Software Requirements (Platform Choice)

- Rust toolchain (stable or nightly, depending on target).
- ESP-IDF for ESP32 development.
- Linux environment with Rust installed.

3.5.3 Hardware Requirements

- ESP32 development board.
- Computer running Linux for development and simulation.

3.6 Analysis Models: SDLC Model to be Applied

- Agile development model with iterative releases.
- Continuous integration and community contributions via GitHub.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

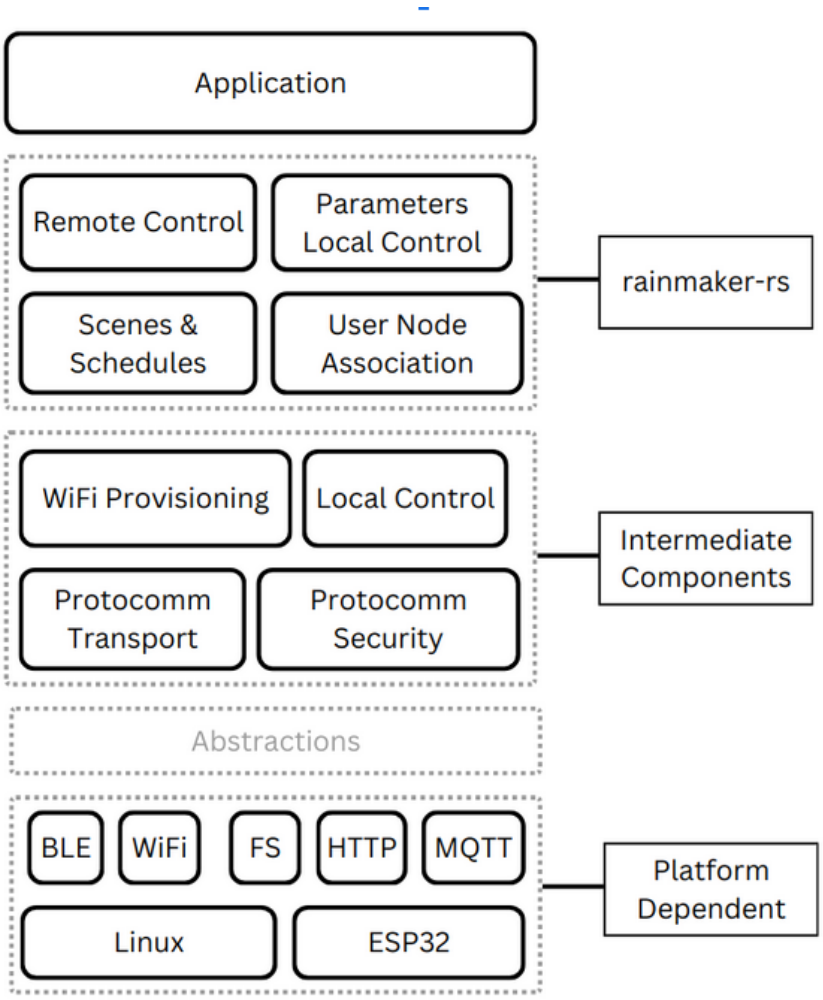


Figure 4.1: Layered System Architecture of RainMaker-rs

Figure 4.1 illustrates the layered architecture of RainMaker-rs. The system is organized into three primary layers:

The system uses a layered architecture:

- **Platform Dependent Abstractions:** Handles specific hardware/OS features (BLE, WiFi, etc.) for ESP32 and Linux.
- **Intermediate Components:** Manages WiFi setup, local control, and secure communication.

- **RainMaker-rs Core:** Contains the main logic for remote control, updates, scheduling, and user-node links.
- **Application Layer:** Your custom code sits here, using the layers below.

4.2 Data Flow Diagrams

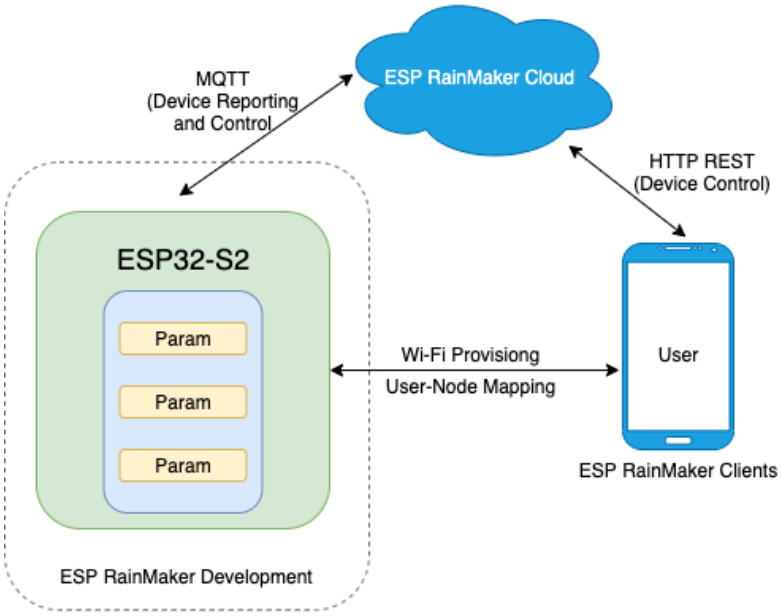


Figure 4.2: Level 0 Data Flow Diagram of RainMaker-rs System

Figure 4.2 shows the Level 0 (context-level) Data Flow Diagram for RainMaker-rs. The Level 0 DFD shows the main interactions:

- **User:** Uses mobile apps/clients to interact via HTTP.
- **Node:** The device running the RainMaker-rs code.
- **ESP RainMaker Cloud:** Handles communication via MQTT.

Data flows include WiFi provisioning from User to ESP32, MQTT messages between ESP32 and Cloud, and HTTP commands from User (via Cloud) to ESP32.

4.3 Entity Relationship Diagram

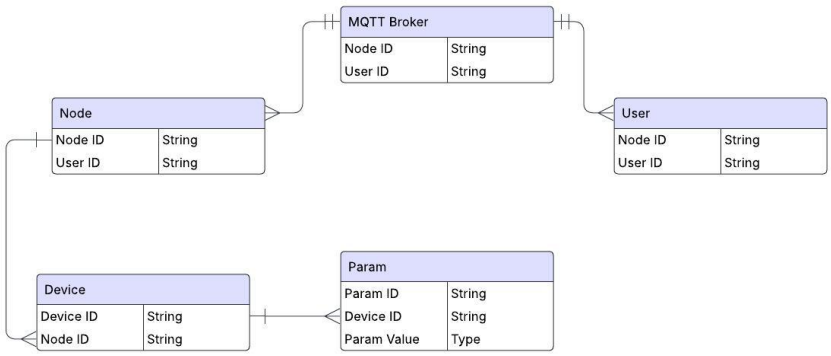


Figure 4.3: Entity Relationship Diagram for RainMaker-rs

Figure 4.3 presents the ER diagram for the RainMaker-rs system. The ER diagram shows how data is structured:

- **User:** Can control multiple Nodes.
- **Node:** A group of Devices, linked to one User.
- **Device:** Belongs to a Node and has multiple Parameters.
- **Param:** A controllable/monitorable attribute of a Device.
- **MQTT Broker:** Connects Users and Nodes for messaging.

Each relationship ensures structured interaction between user devices and the RainMaker cloud infrastructure, maintaining a clear mapping for secure, scalable home automation.

4.4 UML Diagram

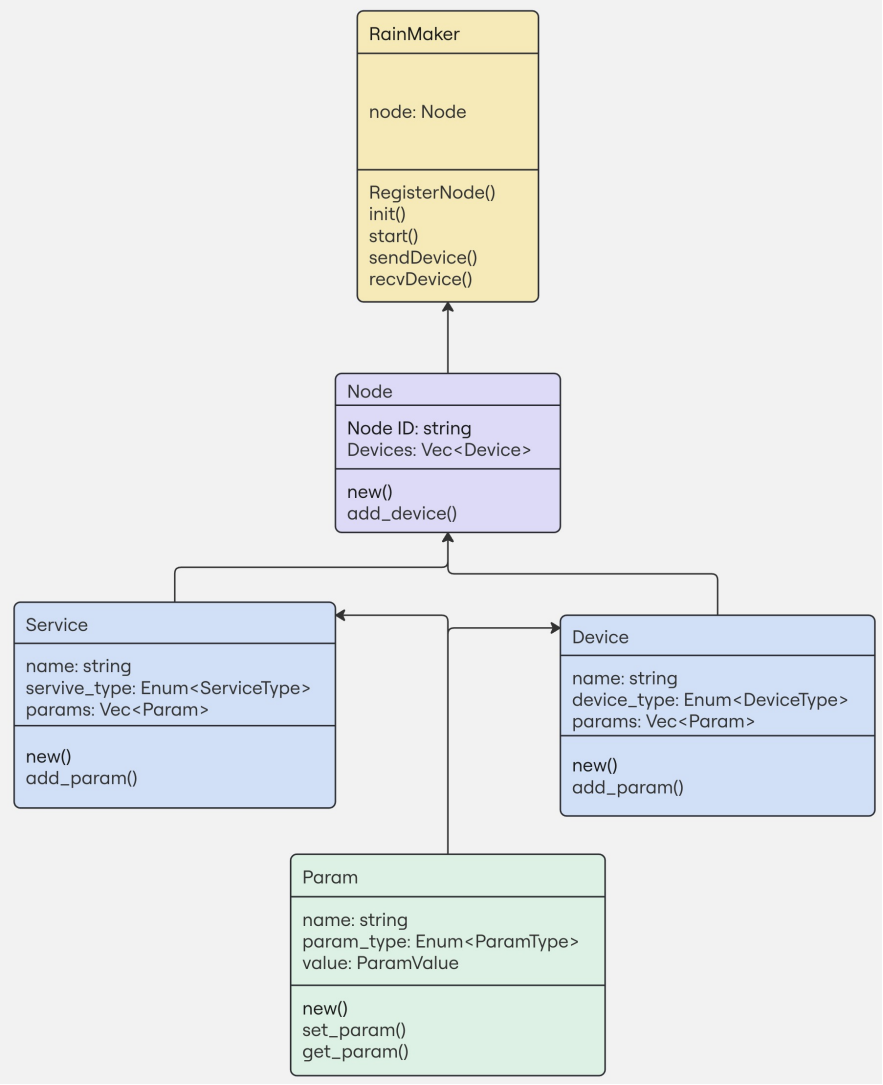


Figure 4.4: Class Diagram of RainMaker-rs Core System

Figure 4.4 illustrates the core class structure of RainMaker-rs.

The class diagram outlines the core software structure:

- **RainMaker:** The main class, managing the Node and system setup/communication.
- **Node:** Holds a list of Devices.

- **Device / Service:** Represent manageable parts of the system, containing Parameters.
- **Param:** Represents a key-value setting with methods to get/set its value.

The design follows a modular, object-oriented structure that promotes flexibility and code reuse. Each class encapsulates its specific functionality and provides public interfaces for interaction.

4.5 Sequence Diagram

4.5.1 Device Parameter Control Flow

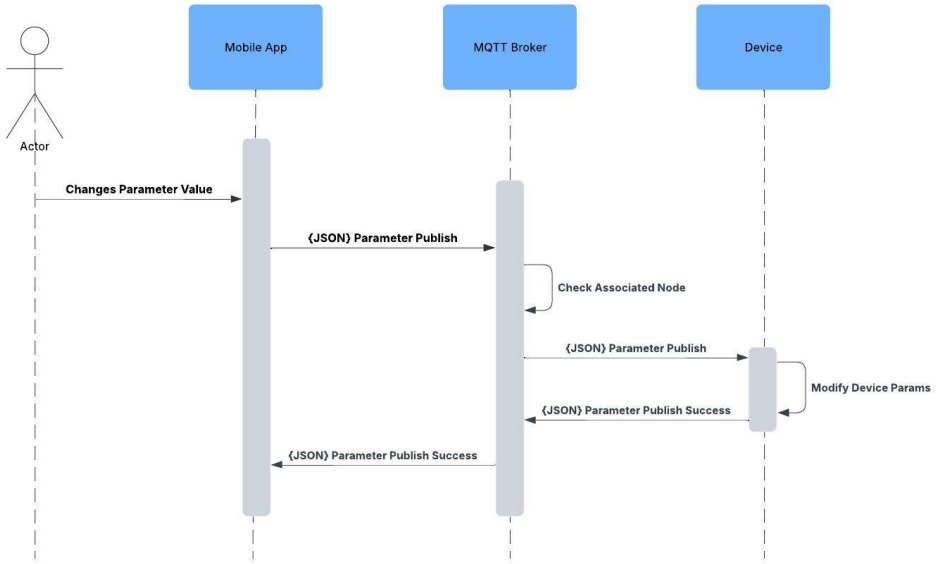


Figure 4.5: Sequence Diagram: Device Parameter Control

Figure 4.5 depicts how a user initiates parameter changes using the mobile app. The sequence is as follows:

1. The user interacts with the mobile application to change a device parameter.
2. The mobile app publishes the updated parameter value as a JSON payload to the MQTT broker.

3. The MQTT broker validates the node association and forwards the update to the respective device.
4. The device modifies the internal parameter and responds with a success message back through the same path.

4.5.2 Device Provisioning Flow

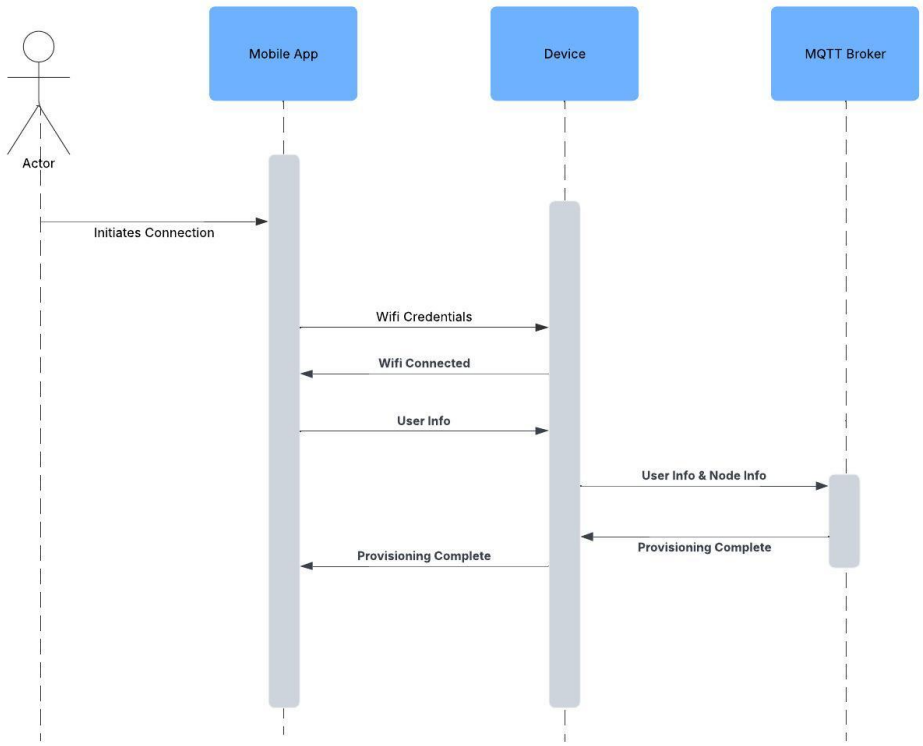


Figure 4.6: Sequence Diagram: Device Provisioning

Figure 4.6 illustrates the flow of provisioning an ESP device:

1. The user initiates the provisioning flow via the mobile app.
2. Wi-Fi credentials are sent to the ESP device.
3. Upon successful connection to Wi-Fi, the device shares user and node information with the MQTT broker.
4. The MQTT broker completes provisioning and sends an acknowledgment back through the device to the app.

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

Project estimation was done primarily in terms of effort (person-hours), timeline, and resource allocation. Since this was a student-led project with no external funding, the focus was on efficient time management and leveraging freely available tools and open-source libraries.

5.1.1 Reconciled Estimates

Initial estimates were made at the beginning of the project based on the complexity of tasks and available documentation for ESP-IDF and Rust embedded ecosystems. However, since `rainmaker-rs` involved platform abstraction, protocol compatibility, and developing from scratch in Rust, actual time taken varied in some areas.

- **Initial Estimate:** 8 months of development, 3 developers contributing 6–8 hours/week.
- **Actual Effort:** Due to the exploratory nature of the project and debugging challenges with embedded Rust, actual effort averaged 8–10 hours/week per person over 9 months.
- **Major Overruns:** Time spent in BLE provisioning, OTA firmware updates, and handling quirks of third-party crates exceeded expectations.
- **Underestimated Tasks:** Porting the SoftAP-based protocomm protocol and ensuring seamless local + cloud integration were more complex than initially anticipated.

Despite these variations, all major objectives were completed within one academic year through consistent effort and iterative refinement.

5.1.2 Project Resources

The project was executed using readily available hardware and open-source tooling. Resources used are listed below:

- **Hardware:**
  - ESP32-WROOM-32 development boards (x2)
  - USB-to-Serial cables and a multimeter for debugging

  - Smartphone (for provisioning via RainMaker mobile app)

- **Software:**
  - Rust (with `no_std` support, and ESP-specific crates)
  - ESP-IDF (reference only)
  - Git and GitHub for version control and collaboration
  - VS Code + rust-analyzer for development
  - Linux environment for local testing

- **Documentation and References:**
  - Espressif official documentation (ESP-IDF, RainMaker)
  - Rust Embedded Book
  - Community forums and GitHub discussions

All resources were selected based on cost-efficiency, compatibility with Rust, and suitability for embedded systems development. The project did not incur any significant monetary costs and was carried out using personal computing equipment.

5.2 Risk Management

Effective risk management was essential for the successful development and delivery of the `rainmaker-rs` project. Risks were identified at each phase of the project, analyzed for severity and likelihood, and strategies were developed to mitigate or monitor them.

5.2.1 Risk Identification

Potential risks were identified at the start of each development cycle and continuously monitored throughout the project. These risks spanned across technical, resource-related, and coordination domains. The key identified risks included:

- **Unfamiliarity with Rust for embedded systems:** Rust's ecosystem for embedded development, especially targeting ESP32, is not as mature as C-based frameworks.
- **Limited hardware access:** Hardware unavailability or sharing delays could block progress on integration and testing phases.

- **Protocol compatibility:** Ensuring compatibility with existing ESP RainMaker C APIs and cloud services could introduce integration risks.
- **Dependency on third-party crates:** Community-supported Rust crates for Wi-Fi, MQTT, and HTTP may have bugs or lack necessary features.
- **Timeline overrun:** Due to the scope and ambition of the project, there was risk of certain modules taking longer than planned.

5.2.2 Risk Analysis

Each risk was analyzed based on two parameters: the likelihood of occurrence and the impact on the project. A risk matrix was created internally to prioritize issues needing early mitigation:

- **High Likelihood, High Impact:** Rust embedded compatibility issues; mitigated through early prototyping.
- **Medium Likelihood, High Impact:** Protocol interoperability and SoftAP/BLE provisioning; addressed via detailed reading of ESP-IDF sources.
- **Low Likelihood, High Impact:** Hardware failure or ESP32 module malfunctions; mitigated by having backup hardware.
- **Medium Likelihood, Medium Impact:** Unavailable maintainers of dependent crates; mitigated by contributing patches or building in-house fallbacks.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

To handle the identified risks, the following strategies were adopted:

- **Mitigation:** Initial stages focused on exploratory PoC work and modular design to decouple high-risk components.
- **Monitoring:** Weekly sync-ups were held to monitor technical blockers or resource constraints.
- **Management:** Contingency plans (such as fallback to C-based reference for comparison) were set up to ensure forward progress even if full Rust abstraction became infeasible.

Overall, active risk tracking and timely communication helped the team deliver all core functionalities within the planned timeline despite working in an evolving and less-documented technology space.

5.3 Project Schedule

5.3.1 Project Task Set

- July-August: Literature survery, Explore RainMaker C implementation
- September: Low Level Abstractions(WiFi, MQTT, Protocomm, NVS)
- October: Initial PoC, Remote Control
- November: Protocomm SoftAP transport implmentation
- December: Protocomm Sec1, WiFi provisioning and user-node association
- January: Protocomm BLE transport
- February: Code cleanup and publishing cargo crate, Local control
- March: OTA updates, Scenes and Schedules

5.3.2 Timeline Chart

5.4 Team Organization

The `rainmaker-rs` project was executed by a team of three students, each contributing across various modules based on their strengths and interest areas. The team adopted an agile, collaborative approach to development, where responsibilities evolved dynamically as the project progressed.

5.4.1 Team Structure

Given the exploratory nature of the project and the relatively small team size, a flat team structure was adopted. While all team members were involved in development, responsibilities were broadly divided as follows:

- **Team Member Chinmay:** Focused on implementing low-level abstractions for Wi-Fi, MQTT, and NVS in Rust.

Table 5.1: Timeline Chart

Column 1	**Column 2**
Literature Survey	July 22 - Aug 16
Understanding ESP RainMaker C implementation	Aug 20 - Aug 31 2
Cross Platform Abstractions (WiFi, MQTT, Protocomm, NVS)	Sep 9 - Sep 25
Proof of Concept	Sep 27 - Oct 8
Remote Control	Oct 9 - Oct 30
Protocomm SoftAP Transport Implementation	Nov 1 - Nov 20
Protocomm Security Implementation	Nov 25 - Dec 13
WiFi Provisioning and User Node Association	Dec 16 - Jan 10
Protocommm BLE Transport Implementation	Jan 13 - Jan 31
Publishing rainmaker-rs as a Cargo crate	Feb 3 - Feb 10
Local Control	Feb 14 - Feb 28
Over-The-Air Updates	Mar 3 - Mar 23
Scenes and Schedules	Mar 25 - Apr 3

- **Team Member Akshay:** Worked on device modeling, parameter management, and implementing the node-device-param structure.
- **Team Member Shreyash:** Handled integration, protocomm (SoftAP and BLE) provisioning mechanisms, and overall system architecture.

The modular architecture of `rainmaker-rs` enabled parallel work with minimal blocking. Roles and ownership were fluid — tasks were taken up based on readiness and workload balance.

5.4.2 Management Reporting and Communication

Since this was a student-led project, internal reporting and communication formed the backbone of project management. Key practices included:

- **Weekly check-ins:** Regular virtual or in-person meetings were held to assess progress, address blockers, and reallocate tasks as necessary.

- **Progress reporting:** Each team member maintained logs of weekly contributions and decisions. These were used for internal review and compiling the final report.
- **Issue-based discussion:** GitHub issues and pull requests were the primary medium for technical discussions and decisions.

This lightweight yet effective communication process ensured transparency, timely updates, and alignment among team members throughout the development lifecycle.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **Remote Control**
  Remote control in ESP RainMaker enables users to operate and monitor their ESP32-based IoT devices from anywhere over the internet. Once a device is connected to the Espressif cloud and linked to a user account via the ESP RainMaker app, users can control it without being on the same local Wi-Fi network. This allows seamless operation of smart home appliances, lighting, sensors, and other connected devices from virtually any location. The feature relies on secure cloud communication using protocols like MQTT over TLS and uses OAuth 2.0 for authentication, ensuring that only authorized users can access and control the devices. Unlike traditional systems, there is no need for port forwarding, static IPs, or additional networking configuration.

  ESP RainMaker's remote control is designed to be simple for users and developers alike. From the user's perspective, the app provides real-time updates and control options, while developers benefit from built-in cloud support and secure communication without managing their own infrastructure.

  Overall, remote control in ESP RainMaker enhances flexibility, convenience, and security in IoT applications, making it ideal for smart homes, industrial automation, and remote monitoring systems. It offers a truly connected experience with minimal setup and strong security by default.

- **WiFi Provisioning**
  Wi-Fi provisioning in ESP RainMaker refers to the process of securely connecting an ESP32-based IoT device to a Wi-Fi network, enabling it to communicate with the internet and the RainMaker cloud. This is a crucial first step in setting up any RainMaker-enabled device, as it allows the device to become accessible for control, monitoring, and updates via the mobile app or cloud services.

  The provisioning process is user-friendly and designed for non-technical users. It typically involves the mobile app guiding the user through connecting the device to their local Wi-Fi network. ESP RainMaker uses Bluetooth Low Energy (BLE) or SoftAP modes for initial communication between the phone and the device. During provisioning, the mobile app securely transmits the Wi-Fi credentials to the device, which then connects to the specified network and registers itself with the Espressif cloud.

Security is a key part of Wi-Fi provisioning. The communication between the app and the device during setup is encrypted, ensuring that sensitive information like Wi-Fi passwords remains protected. Once the device is successfully provisioned, it appears in the user's account within the RainMaker app, ready for remote control and configuration. Wi-Fi provisioning in ESP RainMaker simplifies onboarding, ensuring a secure and seamless user experience for IoT deployments.

- **User Node Association**
  User node association in ESP RainMaker refers to the process of linking an IoT device (node) to a specific user account, granting that user control and access over the device. This association ensures that only authorized users can interact with the device and its functionalities, enhancing security and personalization.

  When a device is provisioned and connected to the ESP RainMaker cloud, the user initiates the association by logging into the **ESP RainMaker app**. During this process, the app links the device (node) to the user's account, allowing them to manage and control it. The device's state, configuration, and settings are then synchronized with the user's profile, ensuring a personalized experience for each user.

  User node association is crucial for managing multiple devices in a home or industrial setup. It allows each user to have their own set of devices that they can control, and it ensures that access to those devices is restricted based on the authenticated user. This feature also supports multi-user setups, where different users may have different levels of control or access to various devices.

  In summary, user node association in ESP RainMaker simplifies device management, secures user access, and enables personalized interactions with IoT devices, making it essential for seamless and secure IoT deployments.

- **Local Control**
  Local control in ESP RainMaker allows users to manage their ESP32-based IoT devices directly over the local Wi-Fi network, without relying on the cloud or internet connection. This feature is ideal for scenarios where users need immediate, low-latency control of their devices, such as in smart homes or automation systems.

  When a device is connected to the same Wi-Fi network as the user's mobile app, local control enables fast communication between the app and the device, bypassing the cloud. The app sends commands directly

to the device over the local network, ensuring real-time responsiveness and lower latency compared to cloud-based control. This is particularly useful when users are at home or within range of their network and want to avoid the slight delays that can occur with cloud communication.

Local control also provides a layer of reliability, as it does not depend on the internet or cloud infrastructure. Even if the internet connection is down, users can still control their devices as long as they are on the same local network.

Overall, local control in ESP RainMaker offers a fast, secure, and reliable way to interact with IoT devices within a home or facility, enhancing user experience and system performance.

6.2 Tools and Technologies Used

- **Rust Programming Language**
  Rust is a systems programming language designed for safety, performance, and concurrency. It was created to overcome the challenges of managing memory safely without sacrificing performance, which is often a concern in low-level programming languages like C and C++.

  One of Rust's key features is memory safety. Rust enforces strict rules about how memory is accessed, preventing common programming errors such as null pointer dereferencing, buffer overflows, and data races. The language achieves this with a system of ownership, borrowing, and lifetimes, which ensures that data is accessed in a controlled and safe manner at compile-time, eliminating the need for a garbage collector.

  Rust is also built for high performance, making it a great choice for system-level programming, game development, web assembly, and other performance-critical applications. It generates highly efficient machine code that rivals C and C++ in terms of speed, but with better safety guarantees.

  Another standout feature of Rust is its support for concurrency. Rust's ownership system ensures that multiple threads can safely operate without the risk of data races, making it easier to write concurrent programs without introducing bugs.

  Rust's strong and modern tooling, such as the Cargo package manager and built-in testing framework, provides a great developer experience, making it easy to manage dependencies, build projects, and run tests.

- **Cargo Package Manager**
  Cargo is the package manager and build system for the Rust programming language. It is an essential tool that helps manage Rust projects, including handling dependencies, building code, running tests, and packaging the final application for distribution.

  With Cargo, developers can easily specify the libraries (or crates) their project depends on, and Cargo will automatically download, compile, and manage those dependencies. This is especially helpful when working with external libraries or when creating reusable components. Cargo handles all of the complexities involved in fetching dependencies, ensuring compatibility, and resolving version conflicts, making it much easier to work on large projects.

  In addition to dependency management, Cargo also provides a streamlined process for compiling and running Rust projects. It automates tasks such as building the project, running tests, and generating documentation, saving developers time and effort. Cargo also allows users to publish their projects to crates.io, Rust's official package registry, making it easier to share and distribute Rust libraries and applications.

  Cargo's key benefits include its simplicity, efficiency, and integration with the wider Rust ecosystem, ensuring that developers can focus on writing high-quality code while Cargo handles the underlying package management and build processes.

CHAPTER 7
SOFTWARE TESTING

7.1 Type of Testing

Given the nature of the project: a cross-platform library designed to run both on embedded (ESP32) and desktop (Linux) environments: the primary form of testing was functional validation across both platforms. Although formal testing strategies such as unit testing or automated test coverage measurement were not systematically implemented, the team ensured that all features behaved consistently across targets.

Types of testing that were implicitly or partially conducted include:

- **Functional Testing:** The functionality of each component (e.g., provisioning, MQTT communication, device control) was validated manually through practical usage on ESP32 and simulation on Linux.
- **Cross-Platform Testing:** The core abstractions for Wi-Fi, MQTT, NVS, and Protocomm were tested for correctness and stability on both Linux and ESP32, ensuring platform compatibility.
- **Integration Testing:** Once individual components (e.g., parameter management, node-device structure) were implemented, they were integrated into end-to-end flows like provisioning, control, and OTA, and tested as a whole.
- **Exploratory Testing:** During development, features were frequently tested under different usage scenarios to uncover unexpected behaviors.
- **Manual Regression Testing:** After each major change, previously implemented flows were manually re-verified to ensure that no regressions were introduced.

7.2 Test Cases & Test Results

While we did not maintain a formalized test suite, our development workflow ensured comprehensive coverage of major functionalities through manual test cases executed repeatedly across both supported platforms. The following summarizes the key areas tested:

- **Device Provisioning (SoftAP, BLE):** Verified that the ESP32 device could be provisioned using both SoftAP and BLE transports, and that the provisioned credentials led to successful MQTT connections.

- **MQTT Communication:** Confirmed bi-directional parameter updates using JSON payloads between mobile clients and devices through MQTT broker.
- **Parameter Control:** Validated that parameter values (e.g., brightness, power state) were correctly set and retrieved via both REST and MQTT paths.
- **Local Execution:** Confirmed that all abstractions correctly compiled and executed on Linux, enabling effective simulation and development before flashing to ESP32.
- **Scenes and Schedules:** Verified correct execution of parameter changes triggered by schedule and scene configuration.
- **OTA Updates:** Tested the over-the-air firmware update flow to ensure devices could receive and apply new firmware versions.

Although automated test frameworks (e.g., CI/CD pipelines, unit tests) were not used in this project, the team ensured correctness and robustness through continuous testing on real hardware and development machines.

CHAPTER 8
RESULTS

8.1 Outcomes

- **Cross-Platform IoT Agent in Rust**: Developed a fully functional ESP RainMaker agent in Rust, enabling seamless operation on both ESP32 microcontrollers and Linux platforms. This broadens the scope of IoT applications beyond the original C-based SDK, which was limited to ESP32 SoCs.
- **Wi-Fi Provisioning Without Hardcoding**: Implemented dynamic Wi-Fi provisioning, allowing devices to be configured without embedding credentials in the firmware. This enhances security and simplifies the onboarding process for end-users.
- **Remote Device Control via Cloud Integration**: Enabled remote control of IoT devices through integration with the ESP RainMaker cloud, facilitating real-time interactions via mobile applications. This supports various use cases, including home automation and industrial monitoring.
- **User-Node Association and Device Sharing**: Established mechanisms for associating devices (nodes) with specific user accounts, allowing personalized control. Additionally, implemented features for sharing device access among multiple users, promoting collaborative environments.
- **Integration with Voice Assistants**: Achieved compatibility with popular voice assistants like Amazon Alexa and Google Home, enabling voice-controlled operations of connected devices. This enhances user convenience and accessibility.
- **Modular and Extensible Architecture**: Designed the system with a modular architecture, facilitating easy integration of additional features and support for other microcontrollers in the future. This ensures scalability and adaptability to evolving IoT requirements.
- **Comprehensive Documentation and Examples**: Provided thorough documentation and example applications to assist developers in understanding and utilizing the `rainmaker-rs` framework effectively. This supports community engagement and accelerates development cycles.
- **Open Source Contribution and Community Engagement**: Released the project under open-source licenses, encouraging contribu-

tions from the developer community. This fosters collaborative development and continuous improvement of the platform.

8.2 Screen Shots

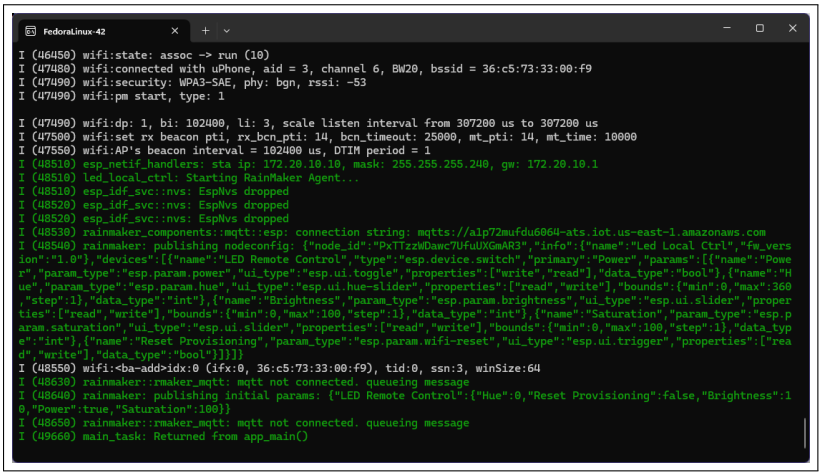


Figure 8.1: Logs running on ESP32C3

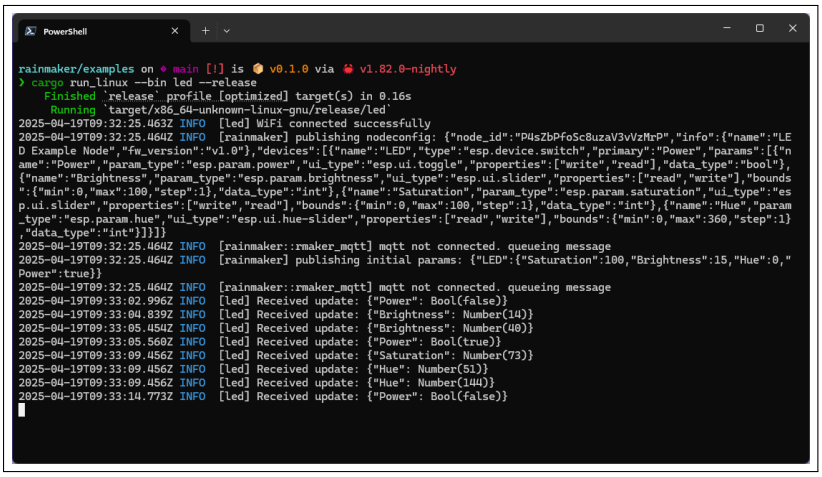


Figure 8.2: Logs running on Fedora Linux

CHAPTER 9
CONCLUSIONS

9.1 Conclusion

The `rainmaker-rs` project was developed to address the limitations of the original C-based ESP RainMaker SDK, which was confined to ESP32 SoCs. By reimplementing the RainMaker agent in Rust, the project achieves cross-platform compatibility, extending support to Linux systems and enhancing the development of IoT applications in home automation and remote device control.

Key achievements of the project include:

- **Cross-Platform Support**: Enabled the RainMaker agent to operate seamlessly on both ESP32 microcontrollers and Linux platforms, broadening the scope of IoT applications.
- **Rust-Based Implementation**: Leveraged Rust's safety and concurrency features to create a robust and efficient RainMaker agent.
- **Modular Architecture**: Designed a modular system facilitating easy integration of additional features and support for other microcontrollers in the future.
- **Enhanced Security**: Implemented secure Wi-Fi provisioning and device control mechanisms, ensuring safe and reliable operation.

9.2 Future Work

The `rainmaker-rs` project has outlined several areas for future development to enhance its capabilities:

- **Local Control**: Implement functionality to control devices within the same local network without relying on cloud services.
- **Over-The-Air (OTA) Updates**: Enable remote firmware updates to simplify maintenance and deployment of new features.
- **Assisted Claiming**: Develop a streamlined device onboarding process using mobile applications to facilitate user-friendly setup.
- **Expanded Platform Support**: Extend compatibility to additional microcontrollers and operating systems, further broadening the project's applicability.

9.3 Applications

The `rainmaker-rs` project has potential applications across various domains:

- **Home Automation**: Facilitate the development of smart home devices that can be controlled remotely, enhancing convenience and energy efficiency.
- **Industrial IoT**: Enable monitoring and control of industrial equipment, improving operational efficiency and safety.
- **Healthcare**: Support remote monitoring of medical devices, contributing to better patient care and management.
- **Agriculture**: Assist in the automation of agricultural processes, such as irrigation and climate control, optimizing resource usage.
- **Research and Development**: Provide a flexible platform for developing and testing new IoT solutions in various research settings.

CHAPTER 10
APPENDIX

10.1 Appendix A

The primary objective of the `rainmaker-rs` project is to reimplement the ESP RainMaker platform in Rust with cross-platform support for Linux and ESP32 systems. While the project primarily involves systems programming and embedded abstraction, we can assess feasibility using theoretical computational concepts for completeness and rigor.

Feasibility in Terms of Decidability and Complexity

The core problems addressed by this project — such as:

- abstracting low-level hardware interfaces (Wi-Fi, NVS, MQTT),
- modeling devices, parameters, and nodes,
- implementing provisioning protocols (SoftAP/BLE),
- supporting scheduling and OTA updates,

are all within the class of **P** (polynomial time) problems, as they do not involve intractable search or optimization spaces. These are primarily algorithmic and protocol-handling tasks that execute in deterministic time bounded by system resources.

Comparison with NP-Complete Problems

Unlike classical NP-Complete problems (e.g., traveling salesman, satisfiability, knapsack), our system does not involve complex state-space exploration or exponential-time search algorithms. However, if we extended the project to include:

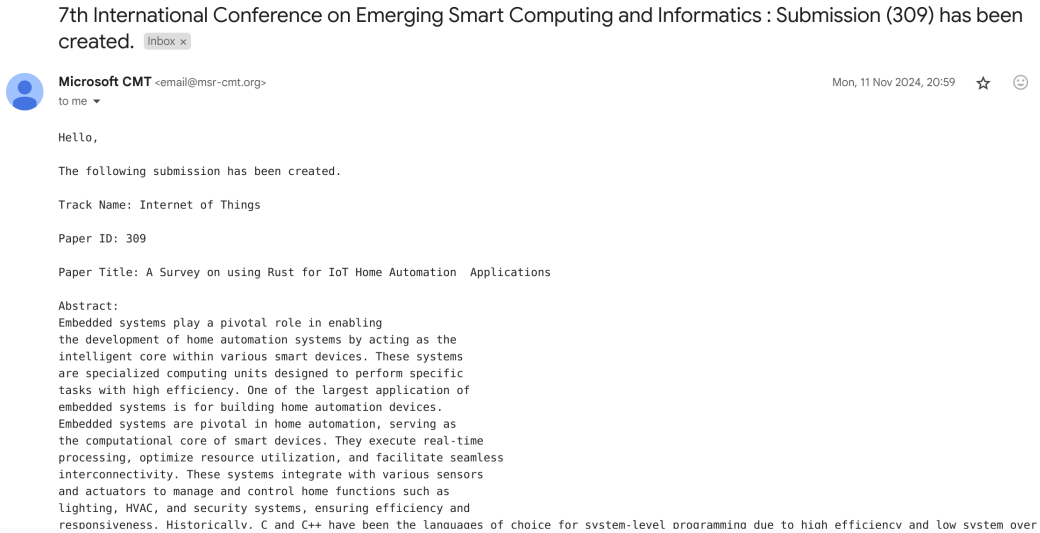
- Optimal task scheduling with resource constraints,
- Heuristic-based OTA rollout planning,
- Predictive control of devices using AI/ML,

then the underlying sub-problems could start falling under NP-Hard or approximation algorithm domains.

10.2 Appendix B

10.2.1 Survey Paper Details

Paper Status: Under Review
Name of Conference: 7th International Conference on Emerging Smart Computing and Informatics [ESCI 2025]
Paper Title: A Survey on using Rust for IoT Home Automation Applications



7th International Conference on Emerging Smart Computing and Informatics : Submission (309) has been created. Inbox ×

Microsoft CMT <email@msr-cmt.org> Mon, 11 Nov 2024, 20:59
to me

Hello,

The following submission has been created.

Track Name: Internet of Things

Paper ID: 309

Paper Title: A Survey on using Rust for IoT Home Automation Applications

Abstract:
Embedded systems play a pivotal role in enabling
the development of home automation systems by acting as the
intelligent core within various smart devices. These systems
are specialized computing units designed to perform specific
tasks with high efficiency. One of the largest application of
embedded systems is for building home automation devices.
Embedded systems are pivotal in home automation, serving as
the computational core of smart devices. They execute real-time
processing, optimize resource utilization, and facilitate seamless
interconnectivity. These systems integrate with various sensors
and actuators to manage and control home functions such as
lighting, HVAC, and security systems, ensuring efficiency and
responsiveness. Historically, C and C++ have been the languages of choice for system-level programming due to high efficiency and low system over

Figure 10.1: Survey Paper: Submission Details

10.2.2 Implementation Paper Details

Paper Status: Under Review
Name of Conference: 9th International Conference on Control Communication, Computing and Automation [ICCUBEA 2025]
Paper Title: rainmaker-rs: A Cross-Platform Rust Implementation of RainMaker for ESP and Linux Devices

9th International Conference on Control Communication, Computing and Automation : Submission (806) has been created. Inbox ×

Microsoft CMT <noreply@msr-cmt.org> 1 May 2025, 10:52 (12 days ago)
to me

Hello,

The following submission has been created.

Track Name: Internet of Things and Computer Networks

Paper ID: 806

Paper Title: rainmaker-rs: A Cross-Platform Rust Implementation of RainMaker for ESP and Linux Devices

Abstract:
RainMaker is a popular platform for building and managing IoT devices. This paper presents rainmaker-rs, a Rust-based implementation of the RainM protocol, designed to be cross-platform and support both ESP microcontrollers and Linux-based systems. Leveraging Rust's safety and performance characteristics, rainmaker-rs provides a robust and efficient solution for IoT device development. The implementation utilizes Cargo.toml compile arguments to facilitate platform-specific builds, enhancing flexibility and maintainability. The project is published on crates.io as rainmaker a rainmaker-components, and its source code is available on GitHub at https://github.com/rainmaker-rs/rainmaker. This paper details the architectur implementation, and performance considerations of rainmaker-rs, demonstrating its efficacy in diverse IoT applications.

Created on: Thu, 01 May 2025 05:22:31 GMT

Last Modified: Thu, 01 May 2025 05:22:31 GMT

Authors:
- akshavlahoti9@gmail.com (Primary)

Figure 10.2: Implementation Paper: Submission Details

10.2.3 Event Participation Details

Event Name: Impetus and Concepts 2025
Domain: Concepts - Embedded / VLSI systems
Position: Third Place

Figure 10.3: Shreyash Bubane Certificate

Figure 10.4: Chinmay Dixit Certificate

Figure 10.5: Akshay Lahoti Certificate

10.3 Appendix C

Document Information

Analyzed document	RainMaker_rs-1.pdf (D209906688)
Submitted	2025-04-22 17:54:00 UTC+02:00
Submitted by	Ratnamala
Submitter email	rspaswan@pict.edu
Similarity	0%
Analysis address	rspaswan.pict@analysis.urkund.com

Sources included in the report

Entire Document

rainmaker-rs: A Cross-Platform Rust Implementation of RainMaker for ESP and Linux Devices Shreyash Bubane
Department
of
Computer Engineering Pune Institute of Computer Technology Pune,
India
bubaneshreyash3@gmail.com
Prof. Ratnamala
Paswan
Department
of
Computer Engineering Pune Institute of Computer Technology Pune, India
rspaswan@pict.edu Chinmay Dixit

Figure 10.6: Plagarism Report: Implementation Paper

Document Information

Analyzed document	BE_Project_Group_80.pdf (D211065890)
Submitted	2025-05-13 11:25:00 UTC+02:00
Submitted by	Ratnamala
Submitter email	rspaswan@pict.edu
Similarity	1%
Analysis address	rspaswan.pict@analysis.urkund.com

Sources included in the report

W	URL: https://blog.industryarc.com/Report/15495/flip-chip-market.html Fetched: 2025-05-13 11:26:00	1
W	URL: https://wikipatents.org/index.php?title=18171318._LOW_POWER-WAKEUP_SIGNAL_FEEDBACK_FOR_WIRELESS_NETWORKS_simplified_abstract_%2528Nokia_Technologies_Oy%2529 Fetched: 2025-05-13 11:26:00	1
W	URL: https://rustrepo.com/repo/material-for-the-course-an-introduction-to-rust Fetched: 2025-05-13 11:26:00	1
W	URL: https://aissmsioit.org/wp-content/uploads/2025/05/ESCI-Report-2024-25.pdf Fetched: 2025-05-13 11:26:00	1
W	URL: https://digitallibrary.usc.edu/asset-management/2A3BF1MADEDED Fetched: 2025-05-13 11:26:00	1
W	URL: https://irjet.net/archives/V7/i6/IRJET-V7I631.pdf Fetched: 2025-05-13 11:26:00	1

Figure 10.7: Plagarism Report: BE Project Report

CHAPTER 11
REFERENCES

[1] Espressif Systems, "Espressif RainMaker," [Online]. Available: https://rainmaker.espressif.com

[2] Espressif Systems, "ESP-IDF Programming Guide," [Online]. Available: https://docs.espressif.com/projects/esp-idf

[3] The Rust Project Developers, "The Rust Programming Language," [Online]. Available: https://www.rust-lang.org

[4] rainmaker-rs Authors, "rainmaker-rs: A Rust Implementation of Espressif's RainMaker," crates.io, [Online]. Available: https://crates.io/crates/rainmaker

[5] A. Pinho, L. Couto, and J. Oliveira, "Towards Rust for Critical Systems," in *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Berlin, Germany, 2019, pp. 19-24, doi: 10.1109/ISSREW.2019.00036.

[6] "Five years of Rust," May 15, 2020. [Online]. Available: https://blog.rust-lang.org/2020/05/15/five-years-of-rust.html

[7] GitHub Blog, "Why Rust is the most admired language among developers." [Online]. Available: https://github.blog/developer-skills/programming-languages-and-frameworks/why-rust-is-the-most-admired-language-among-developers/

[8] Z. Yu, L. Song, and Y. Zhang, "Fearless concurrency? understanding concurrent programming safety in real-world rust software," arXiv preprint arXiv:1902.01906, 2019.

[9] The Embedded Rust Book, "Interoperability." [Online]. Available: https://docs.rust-embedded.org/book/interoperability/

[10] A. Sharma, S. Sharma, S. Torres-Arias, and A. Machiry, "Rust for Embedded Systems: Current State, Challenges and Open Problems," arXiv preprint arXiv:2311.05063, 2023.

[11] G. Hoare, "Microsoft: Rust Is the Industry's 'Best Chance' at Safe Systems Programming," 2020.

[12] TheNewStack.io, "Microsoft: Rust Is the Industry's 'Best Chance' at Safe Systems Programming." [Online]. Available: https://thenewstack.io/microsoft-rust-is-the-industrys-best-chance-at-safe-systems-programming/

[13] H. Zhang, "2022 Review – The adoption of Rust in Business," 2022.

[14] Rust Magazine, "2022 Review – The adoption of Rust in Business," Issue 1. [Online]. Available: https://rustmagazine.org/issue-1/2022-review-the-adoption-of-rust-in-business/

[15] Stefanini, "The Rust Language Revolution: Why Are Companies Migrating?," n.d..

[16] Stefanini, "The Rust Language Technology Revolution: Why Are Companies Migrating?" [Online]. Available: https://stefanini.com/en/insights/news/the-rust-language-technology-revolution-why-are-companies-migrating

[17] A. Levy, M. P. Andersen, B. Campbell, D. Culler, P. Dutta, B. Ghena, P. Levis, and P. Pannuto, "Ownership is theft: Experiences building an embedded OS in Rust," in *Proceedings of the 8th Workshop on Programming Languages and Operating Systems*, 2015, pp. 21-26.

[18] C. Bayılmıs, M. A. Ebleme, Ü. Çavuşoğlu, K. Küçük, and A. Sevin, "A survey on communication protocols and performance evaluations for Internet of Things," *Digital Communications and Networks*, vol. 8, no. 6, pp. 1094-1104, 2022, doi: 10.1016/j.dcan.2022.03.013.

[19] Y. Upadhyay, A. Borole, and D. Dileepan, "MQTT based secured home automation system," in *2016 Symposium on Colossal Data Analysis and Networking (CDAN)*, Indore, India, 2016, pp. 1-4, doi: 10.1109/CDAN.2016.7570945.

[20] The Embedded Rust Book, Figure. [Online]. Available: https://docs.rust-embedded.org/book/assets/crates.png

[21] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of things: A survey on enabling technologies, protocols, and applications," *IEEE communications surveys and tutorials*, vol. 17, no. 4, pp. 2347-76, Jun. 2015.

[22] HiveMQ Blog, "How to Get Started with MQTT." [Online]. Available: https://www.hivemq.com/blog/how-to-get-started-with-mqtt/

[23] Stack Overflow Blog, "What is Rust and why is it so popular," Jan. 20, 2020. [Online]. Available: https://stackoverflow.blog/2020/01/20/what-is-rust-and-why-is-it-so-popular